



Working with an API

10m



© © Getty Images

[Skip to main content](#)

The first task when any developer works with an API is to read the documentation to understand how the API requires you to interact with it. NumbersAPI is very simple. Many are not so simple, and as they will offer greater functionality, the documentation will be more involved.

Example: Salesforce

Look at the documentation for Salesforce APIs.

Salesforce is credited for developing the first web APIs in the early 2000s. This is a RESTful API, but it has 276 methods. Methods such as Holiday (Salesforce, n.d.), which 'represents a period of time during which your customer support team is unavailable. Business hours and escalation rules associated with business hours are suspended during any holidays with which they are affiliated' (para. 1).

In other words, it's a query you make to check when the business in question is closed to customer support.

Example: Google Maps API

One of the most common APIs for web developers to work with is Google Maps API

This API has a different documentation strategy. Rather than presenting you with all the end points and methods, they show by example. They do have a full reference, but the use of examples is a lot easier to understand. Unfortunately, Google now requires people to register a billing account before you can even begin to make requests for maps.

Types of APIs

In the context of APIs, the word 'application' refers to any software with a distinct function. The interface can be thought of as a contract of service between two applications. This contract defines how the two applications communicate with each other using requests and responses.

HTTP is an API, an agreed protocol for browsers to make requests to web applications. It includes POST, PUT, GET and DELETE requests. Other types of API include:

- **Simple Object Access Protocol (SOAP)** – uses XML to allow communication between completely independent applications
- **Service-Oriented Architecture (SOA)** – a service that allows for requests to access varied application services
- **Resource-Oriented Architecture (ROA)** – similar to SOA in that applications can request specific resources; some frameworks, such as Symfony and Ruby on Rails, operate on an ROA basis
- **Representational State Transfer (REST)** – the most popular API type for public APIs today and the type we will be looking at. HTTP is a form of REST API.

In the next section, we will be discussing REST API in detail as it is the most popular and widely used API.

What is a REST API?

A REST API (often also termed a RESTful API) is a protocol to query data or media. We have already introduced this in an earlier topic with the client-side/frontend aspects of a web application. They use HTTP's GET method to retrieve a resource, PUT to change the state of or update a resource (which can be an object, file or block), POST to create a resource and DELETE to remove it.

REST APIs are termed stateless as they do not retain any knowledge of the state of the requesting system. Compare this to a Facebook account, which stores information about the user and retrieves it upon logging in. While you can add things to a specific user via a REST API, the actual request does not need to know who you are; just whether you have permission to make the request you are making.

Most social media platforms offer a REST API to allow developers to get information to display in other applications. When you see someone's Instagram images on their website, the code that requests the Instagram image will have used the Instagram API to fetch the media for a specific account and then displayed that media on the page.

[Skip to main content](#)

Why use REST APIs?

The REST framework was introduced by computer scientist Roy Fielding in 2000, and it changed how content is viewed, modified and transferred online. Many of the most popular web and cloud companies use REST APIs for their applications, including Facebook, YouTube, X (Twitter) and Google.

REST APIs have several benefits, as outlined below.

REST API benefit	Explanation
Flexible	They can handle many types of requests and send data in many different formats.
Scalable	They are designed for communication between any two pieces of software, regardless of size or capability. As a web application grows and adds more resources, its REST API will be able to quickly handle the increasing number and variety of requests.
Incorporate existing web technologies	Incorporating existing web technologies makes them relatively easy to build and use. To request a resource via a REST API, you just need to provide its URL.



Thoughts on this page?

